

Introducción a la Geometría Espacial y Animación 3D

HCI

2026

Objetivos

- Comprender el espacio tridimensional.
- Introducir la geometría espacial aplicada a gráficos 3D.
- Relacionar conceptos matemáticos en el contexto de raylib.
- Comprender el uso de la cámara en escenas 3D.
- Implementar geometría espacial y transformaciones utilizando raylib.

Sistema de Coordenadas 3D

En geometría espacial los objetos existen en:

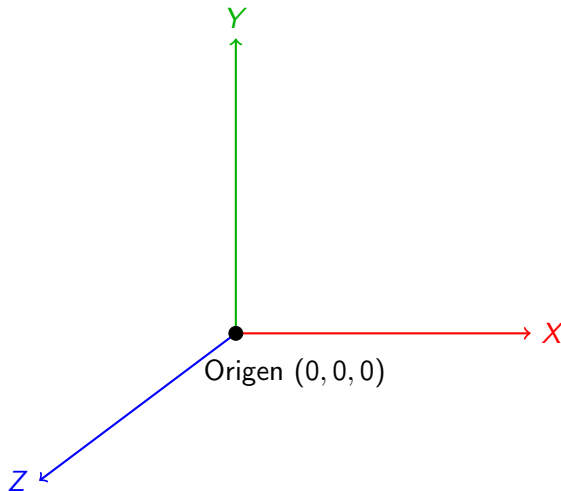
$$(x, y, z)$$

- x representa izquierda/derecha.
- y representa arriba/abajo.
- z representa profundidad.

Vector espacial:

$$\vec{v} = (x, y, z) \in \mathbb{R}^3$$

Representación del Espacio Tridimensional



Esqueleto Básico de un Programa 3D en raylib

```
1 #include "raylib.h"
2
3 int main()
4 {
5     InitWindow(1000, 700, "Escena 3D");
6
7     Camera3D camera = { 0 };
8     SetTargetFPS(60);
9
10    while (!WindowShouldClose())
11    {
12        BeginDrawing();
13        ClearBackground(RAYWHITE);
14
15        BeginMode3D(camera);
16        // Dibujar objetos 3D
17        EndMode3D();
18
19        EndDrawing();
20    }
21
22    CloseWindow();
23    return 0;
24 }
```

Interpretación del Ciclo de Animación 3D

El programa funciona como un ciclo continuo de simulación y renderizado.

1. Inicialización

Se crean los elementos principales de la aplicación:

- ventana,
- cámara,
- escena,
- variables de animación.

2. Bucle principal

El ciclo:

```
while(!WindowShouldClose())
```

mantiene activa la aplicación y permite actualizar continuamente la escena.

Actualización y Renderizado

3. Actualización

En esta etapa se modifican:

- posiciones,
- velocidades,
- rotaciones,
- escalamiento,
- animaciones.

4. Renderizado

La GPU vuelve a dibujar toda la escena tridimensional.

Este proceso ocurre repetidamente para producir la ilusión de movimiento continuo.

Normalmente la animación se ejecuta a:

3D en raylib

```
1 #include "raylib.h"
2
3 int main()
4 {
5     InitWindow(1000, 700, "Sistema 3D");
6
7     Camera3D camera = { 0 };
8
9     camera.position = { 8.0f, 8.0f, 8.0f };
10    camera.target = { 0.0f, 0.0f, 0.0f };
11    camera.up = { 0.0f, 1.0f, 0.0f };
12
13    SetTargetFPS(60);
14 }
```


Interpretación Matemática de la Cámara

La cámara posee una posición espacial:

$$\vec{p} = (x, y, z)$$

También posee una dirección visual:

$$\vec{d} = \textit{target} - \textit{position}$$

Esto permite construir la perspectiva del observador.

Elementos Matemáticos de los Cuerpos 3D

Los cuerpos geométricos tridimensionales poseen propiedades matemáticas que permiten describir:

- dimensiones,
- volumen,
- superficie,
- posición espacial.

Cubo

$$V = l^3$$

- V representa el volumen.
- l representa la longitud de cada lado.

Esfera

$$V = \frac{4}{3}\pi r^3$$

Elementos Matemáticos del Cilindro

Cilindro

El volumen de un cilindro se calcula mediante:

$$V = \pi r^2 h$$

donde:

- V representa el volumen.
- r representa el radio de la base.
- h representa la altura.

En gráficos 3D, estos parámetros permiten construir objetos geométricos con dimensiones controladas matemáticamente.

En raylib, el cilindro puede modelarse especificando:

- posición,
- radio,
- altura,
- número de segmentos.

Transformaciones Geométricas

Las transformaciones permiten modificar objetos en el espacio.

Principales transformaciones:

- Traslación.
- Rotación.
- Escalamiento.

Traslación

Mover un objeto en el espacio.

$$\vec{p}' = \vec{p} + \vec{t}$$

Donde:

- $\vec{p}$ es la posición original.
- $\vec{t}$ es el vector de traslación.

Traslación en raylib

```
1 float x = -5.0f;  
2  
3 x += 0.03f;  
4  
5 DrawCube(  
6     { x, 1.0f, 0.0f },  
7     2.0f,  
8     2.0f,  
9     2.0f,  
10    BLUE  
11 );
```

Rotación

La rotación modifica la orientación de un objeto.

Rotación sobre el eje Y:

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

Rotación en raylib

```
1 rlPushMatrix();
2
3 rlTranslatef(0.0f, 1.0f, 0.0f);
4
5 rlRotatef(
6     angulo,
7     0.0f,
8     1.0f,
9     0.0f
10 );
11
12 DrawCube(
13     {0.0f, 0.0f, 0.0f},
14     2.0f,
15     2.0f,
16     2.0f,
17     ORANGE
18 );
```



Funciones de Rotación en raylib

Para rotar objetos en raylib se utilizan funciones de la biblioteca `rlgl`.

1. `rlPushMatrix()`

Guarda temporalmente el sistema de transformación actual.

Esto permite aplicar transformaciones a un objeto sin modificar el resto de la escena.

2. `rlTranslatef(x,y,z)`

Desplaza el sistema de referencia hacia una nueva posición:

$$(x, y, z)$$

Este desplazamiento define el punto donde se realizará la rotación.

Funciones de Rotación en raylib

3. `rlRotatef(angulo,x,y,z)`

Aplica una rotación sobre un eje espacial.

Parámetros:

- `angulo`: cantidad de rotación en grados.
- `(x,y,z)`: eje de rotación.

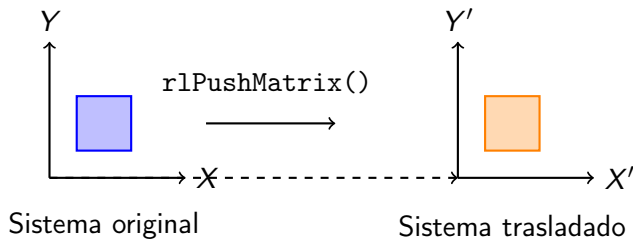
Ejemplo:

$(0, 1, 0)$

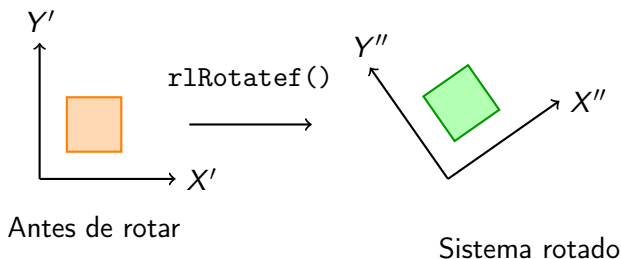
indica rotación sobre el eje Y .

4. `rlPopMatrix()`

Restaura el sistema de transformación original de la escena.

Idea Visual de `rlPushMatrix()` y `rlTranslatef()`

- `rlPushMatrix()` guarda el sistema de referencia original.
- `rlTranslatef(x,y,z)` desplaza el sistema de coordenadas hacia una nueva posición.
- El objeto ahora se dibuja respecto al nuevo sistema trasladado.

Idea Visual de `rlRotatef()` y `rlPopMatrix()`

- `rlRotatef()` rota el sistema de referencia del objeto.
- La figura gira respecto al eje especificado.
- `rlPopMatrix()` restaura el sistema original de la escena.

Escalamiento Geométrico

El escalamiento modifica el tamaño de un objeto tridimensional.

Geométricamente, cada coordenada del objeto se multiplica por un factor de escala.

$$\vec{p}' = s\vec{p}$$

donde:

- $\vec{p}$ representa la posición original del punto.
- $\vec{p}'$ representa la nueva posición escalada.
- s representa el factor de escala.

Interpretación:

- si $s > 1$, el objeto crece.
- si $0 < s < 1$, el objeto disminuye.

Interpretación Espacial del Escalamiento

Supongamos un cubo de tamaño:

$$2 \times 2 \times 2$$

Si aplicamos:

$$s = 2$$

entonces el nuevo tamaño será:

$$4 \times 4 \times 4$$

En raylib, el escalamiento se logra modificando dinámicamente:

- ancho,
- altura,
- profundidad.

La Cámara en Espacios 3D

La cámara representa:

- el observador,
- el punto de vista,
- la perspectiva.

Elementos principales:

- position.
- target.
- up.
- fovy.

Campo Visual de la Cámara

El parámetro FOV representa:

$FOV = \text{Field Of View}$

- FOV pequeño: efecto zoom.
- FOV grande: efecto panorámico.

Cámara Libre en raylib

```
1 UpdateCamera(  
2     &camera,  
3     CAMERA_FREE  
4 );
```

La cámara puede:

- moverse,
- rotar,
- explorar el espacio.

Animación Temporal

La animación consiste en modificar propiedades en función del tiempo.

Ejemplos:

- movimiento senoidal,
- rebotes,
- órbitas,
- trayectorias.

Movimiento Senoidal

El movimiento senoidal modela oscilaciones periódicas.

Ejemplos físicos:

- péndulos,
- ondas,
- vibraciones,
- movimiento armónico simple.

La posición vertical del objeto cambia según:

$$y(t) = A \sin(\omega t)$$

donde:

- $y(t)$ representa la posición vertical.
- A representa la amplitud máxima.
- ω representa la frecuencia angular.

Interpretación Geométrica del Movimiento Senoidal

La función seno produce un movimiento suave de subida y bajada.

Características:

- cuando:

$$\sin(\omega t) = 1$$

el objeto alcanza la altura máxima.

- cuando:

$$\sin(\omega t) = -1$$

el objeto alcanza la altura mínima.

- el movimiento se repite continuamente.

En raylib, este valor puede utilizarse para modificar dinámicamente la

Movimiento Orbital

El movimiento orbital describe trayectorias circulares alrededor de un centro.

Se modela utilizando funciones trigonométricas:

$$x(t) = r \cos(t)$$

$$z(t) = r \sin(t)$$

donde:

- r representa el radio de la órbita.
- t representa el tiempo.
- $x(t)$ y $z(t)$ representan la posición del objeto.

Interpretación Geométrica del Movimiento Orbital

El objeto se desplaza alrededor de un punto central.

Geométricamente:

- el coseno controla el desplazamiento horizontal.
- el seno controla el desplazamiento en profundidad.

La combinación simultánea de ambas funciones genera una circunferencia.

En gráficos 3D este movimiento se utiliza para:

- planetas,
- satélites,
- cámaras orbitales,
- sistemas de animación.

Animación en raylib

```
1 float t = GetTime();  
2  
3 float x = cosf(t) * 4.0f;  
4 float z = sinf(t) * 4.0f;  
5  
6 DrawSphere(  
7     {x, 1.0f, z},  
8     0.5f,  
9     RED  
10 );
```

Relación entre Matemáticas y Gráficos

En gráficos 3D se utilizan:

- álgebra lineal,
- geometría espacial,
- trigonometría,
- vectores,
- matrices.

Los motores gráficos convierten ecuaciones matemáticas en imágenes animadas.

Conclusiones

- raylib facilita el aprendizaje de gráficos 3D.
- La geometría espacial es fundamental en simulación.
- La cámara define la percepción del espacio.
- Las transformaciones permiten animar objetos.
- Las matemáticas son la base de los gráficos por computador.

Gracias